ShieldAI — Carte de l'architecture C (modules_c)

État du code au 11 octobre 2026. Ce qui est codé est décrit tel quel ; ce qui est prévu est marqué « prévu ».

1. L'essentiel en 30 secondes 🧭

Un paquet réseau arrive sous forme d'**octets bruts**. Le **parser** les lit et remplit une fiche propre (`ParserResult`). La **flow table** regarde à quelle conversation (un *flux*) le paquet appartient, crée ou met à jour la fiche du flux (`FlowEntry`), et la range dans deux structures à la fois :

- un **dict** (table de hachage) pour retrouver un flux très vite ;
- une **liste doublement chaînée** pour savoir quel flux est le plus ancien, et le chasser quand la table est pleine.

À côté, une seconde table retient, pour chaque **IP source**, l'ensemble des **ports distincts** qu'elle a visités (une `PortMap`). C'est la base de la détection de scan.

Ce qui n'existe pas encore : la capture (sockets et threads), le ring partagé vers Python, la liste d'IP à ignorer, l'expiration par le temps, l'agrégateur multi-interfaces. Voir le §7.

2. La carte des modules 🗺️

Module	Fichiers	Rôle en une phrase	Utilise	État
utils	`utils/`	Octets vers entiers (IP 4 octets → u32, port 2 octets → u16)	rien	codé
parser	`parser/`	Transforme les octets d'un paquet en `ParserResult`	rien	codé, 3 fichiers de tests
dict	`dict/`	Table de hachage générique, collisions chaînées, rehash à 75 %	rien	codé, tests

Module	Fichiers	Rôle en une phrase	Utilise	État
portmap	`portmap/`	Bitmap de 65 536 ports (8 Ko) + compteurs	rien	codé
flow	`flows/flow.*`	Clé de flux, données du flux, mise à jour, fonctions fournies au dict	parser, dict	codé
dchained_list	`dchained_list/`	File doublement chaînée de `FlowEntry` (ordre d'ancienneté)	flow.h	codé
flow_table	`flows/flow_table.*`	Chef d'orchestre : `ft_process`	flow, dict, dchained_list, portmap, utils	codé
main.c	`main.c`	« Hello world », en attendant le moteur	rien	stub

Qui dépend de qui. Quatre briques sont autonomes : `parser`, `dict`, `portmap`, `utils`. `flow` s'appuie sur `parser` (il lit un `ParserResult`) et sur `dict` (il fournit les fonctions de hachage, comparaison et copie). `dchained_list` n'a besoin que du **type** `FlowEntry` défini dans `flow.h`. `flow_table` assemble tout.

3. Le voyage d'un paquet 📦

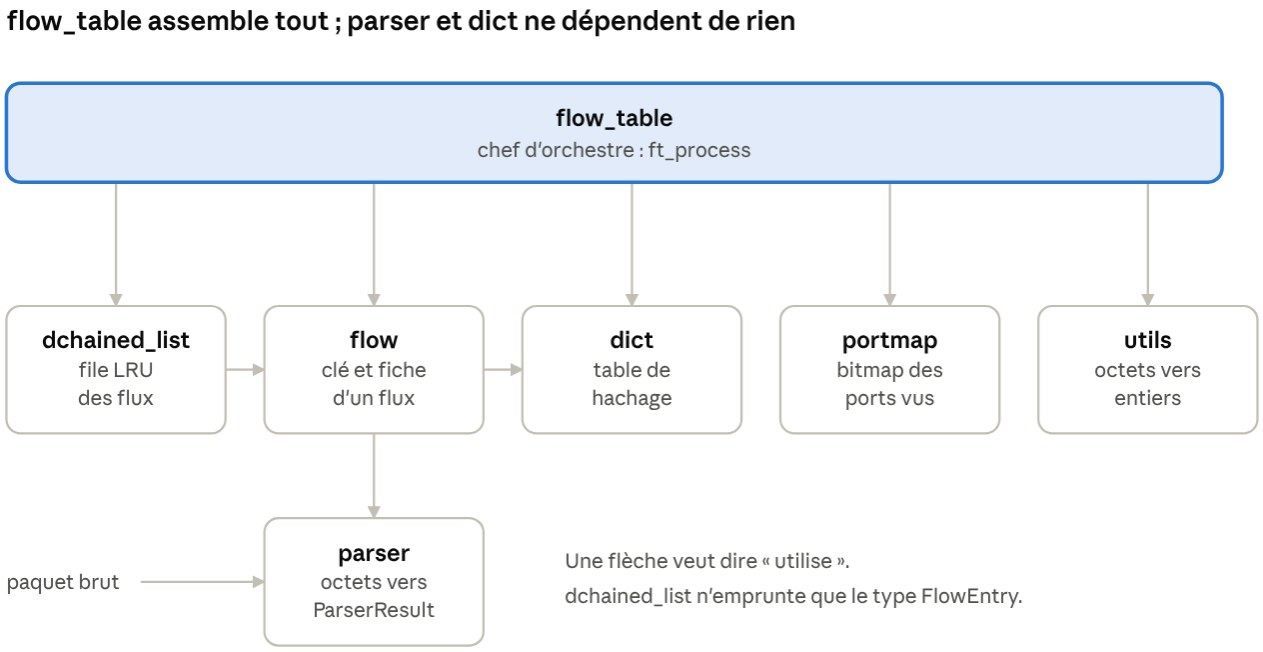
Tout passe par `ft_process(ft, pr)`.

1. Les octets arrivent (`pkt`, `len`) et `parse()` renvoie un `ParserResult`. Si `code` n'est pas `SUCCES`, `ft_process` refuse le paquet.
2. `ft_process` construit une **clé de flux** sur la pile avec `build_flow_key`. La clé est *canonique* : les deux IP et les deux ports sont rangés par ordre (min / max), donc A→B et B→A donnent la même clé.
3. `ft_get` cherche la clé dans `flow_dict` : hachage, bucket, comparaison de la clé complète.
4. **Flux déjà connu** : `entry_update` détermine le sens (ltr si le paquet vient de l'initiateur, rtl sinon), incrémente les compteurs de paquets et de flags TCP, puis le flux passe à la fin de la file (`move_to_end`).

5. **Nouveau flux** :
   - création d'un `FlowEntry` (calloc) et copie de la clé (`flow_key_dup`) ;
   - `build_flow_entry` crée le `FlowData` (calloc) et y met l'initiateur, le receveur, les flags du premier paquet ;
   - `_port_per_ip_add` augmente le `refcount` de l'IP initiatrice dans `port_per_ip` ;
   - `ft_add` évince le plus ancien si la table est pleine, insère dans le dict (`dict_setitem`) et ajoute à la fin de la file (`add_to_end`) ;
   - `ft_port_per_ip_visit` note le port destination dans la `PortMap` de l'IP source.

4. Les schémas 🎨

Schéma A — Les blocs et leurs dépendances

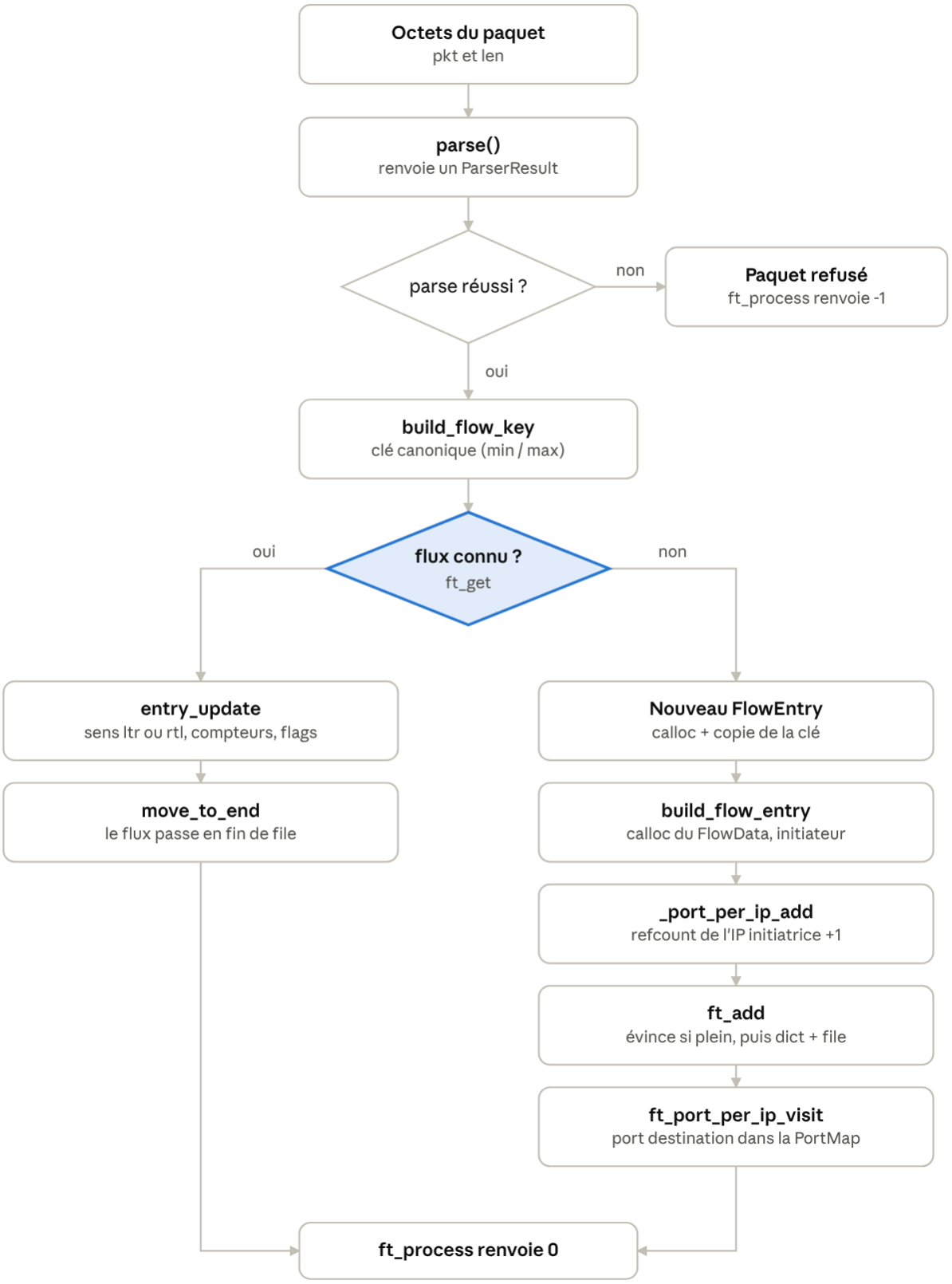


dépendances entre modules · 8 blocs

On lit de haut en bas : chaque flèche veut dire « utilise ». Les briques du bas (`parser` , `dict`) n'ont besoin de personne.

Schéma B — Le voyage d'un paquet

Le chemin d'un paquet se sépare selon que le flux existe déjà

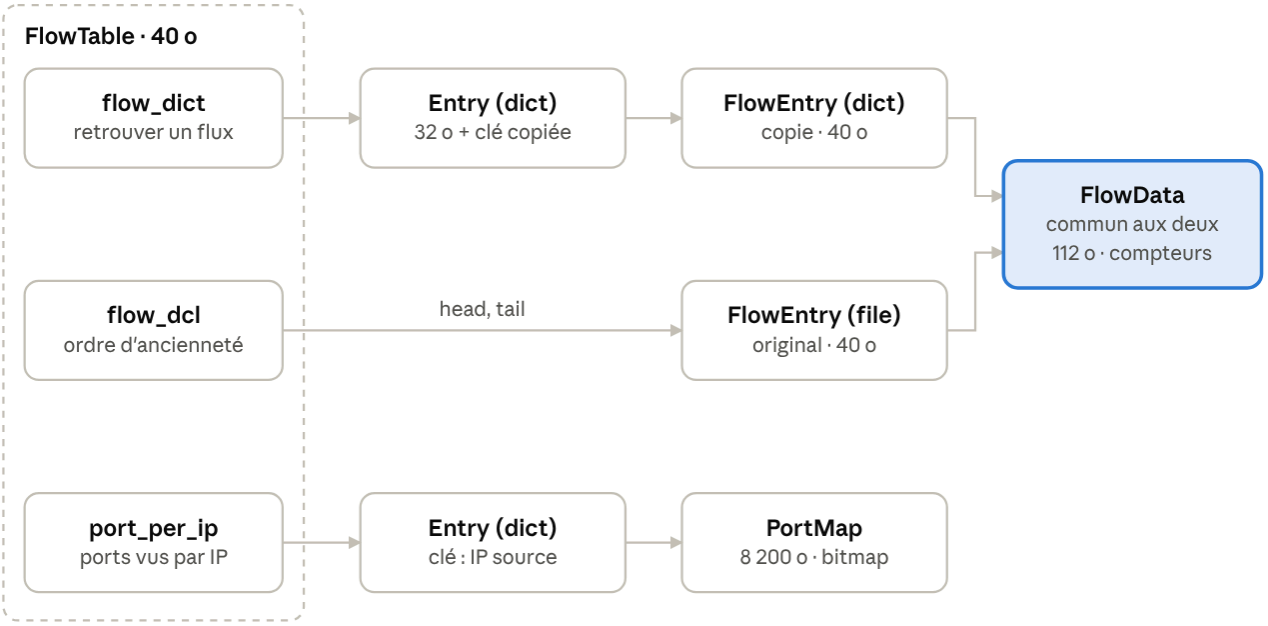


voyage d'un paquet dans ft_process · 1 embranchement

Tous les paquets font le même début de trajet. Le seul vrai embranchement : le flux existe déjà (on met à jour sa fiche) ou non (on crée tout : fiche, données, compteur d'IP, entrée dans le dict et dans la file).

Schéma C — Où vit un flux en mémoire

Un flux existe en deux FlowEntry qui partagent le même FlowData

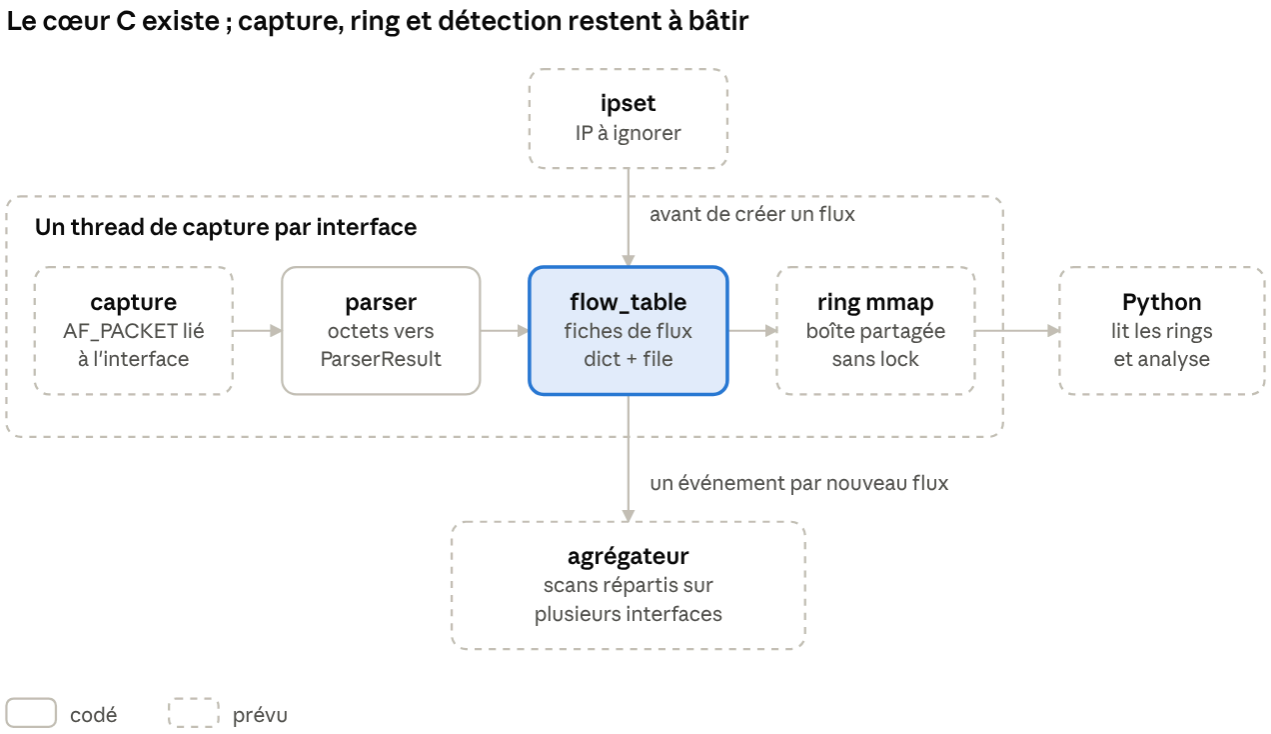


entry->key est partagée par les deux FlowEntry ; l'Entry du dict garde sa propre copie de la clé.
Chaque IP source coûte une PortMap de 8 200 octets, libérée quand son refcount tombe à 0.

objets en mémoire pour un flux et pour une IP · 8 objets

Les deux exemplaires viennent du fait que le dict copie la clé et le `FlowEntry` en les rangeant (`flow_key_dup` et `flow_value_dup`). Seul le `FlowData` n'est pas copié.

Schéma D — La cible (avec le prévu en pointillé)



architecture cible · 2 blocs codés, 5 prévus

En trait plein : ce qui existe déjà. En pointillé : ce que le PDF de notes prévoit. Chaque thread aurait sa propre flow table et son propre ring ; Python les vide à tour de rôle.

5. Qui possède quoi en mémoire 🧠

Sur la pile (disparaît à la fin de la fonction)

- `ParserResult` (92 octets), renvoyé **par valeur** par `parse()` ;
- la clé locale de `ft_process` ;
- la `PortPerIpKey` locale (20 octets).

Sur le tas, une fois par table

- `FlowTable` : 40 octets ;
- `flow_dict` : un `Dict` (72 o) + son tableau de buckets + une `Entry` (32 o) par flux ;
- `flow_dcl` : une `DChainedList` (32 o), qui ne contient que head, tail et des compteurs ;
- `port_per_ip` : un second `Dict`, clé = `PortPerIpKey`, valeur = `PortMap`.

Sur le tas, pour chaque flux

Objet	Taille	Créé par	Libéré par
`FlowEntry` (l'original, celui que la file chaîne)	40 o	`ft_process` (calloc)	`_ft_evict`
`FlowData` (compteurs, adresses, verdict)	112 o	`build_flow_entry` (calloc)	`_ft_evict`
`FlowKey` copiée, pointée par `entry->key`	20 o (v4) / 44 o (v6)	`flow_key_dup` dans `ft_process`	`_ft_evict`
`Entry` du dict + **sa propre copie de clé** + **sa propre copie du `FlowEntry`**	32 o + clé + 40 o	`dict_setitem` (via `key_dup` et `value_dup`)	`dict_delitem` (via `key_free` et `value_free`)

Le point clé de la carte : comme le `flow_dict` est créé avec `flow_key_dup` et `flow_value_dup`, `dict_setitem` fait des copies. Un flux existe donc en **deux exemplaires de `FlowEntry`** : l'original (que la liste chaîne avec `lru_prev` / `lru_next`) et une copie superficielle rangée dans le dict. Les deux pointent vers le **même** `FlowData` et la même clé `entry->key`.

Par IP source

Une `PortMap` pèse **8 200 octets** (8 192 de bitmap + 2 compteurs). Elle est allouée par calloc quand une IP apparaît, et libérée par le dict (`value_free = free`) quand son `refcount` retombe à 0, c'est-à-dire quand le dernier flux de cette IP est évincé. C'est aussi pourquoi le PDF de notes prévoyait plutôt un petit `portset` de 32 ports et gardait la bitmap « pour plus tard ».

6. Détail module par module 🔎

parser

- Entrée : `parse(pkt, len)`. Sortie : un `ParserResult` par valeur, mis à zéro avec `memset` avant remplissage.
- Couches gérées : Ethernet, VLAN et QinQ (4 tags maximum), IPv4, IPv6 (avec en-têtes d'extension, 8 maximum), TCP, UDP, ICMP / ICMPv6, ARP.

- Codes de retour : `SUCCES`, `ERROR` (argument invalide), `TOO_LITTLE` (paquet trop court), `INCOHERENCE` (le paquet se contredit), `NOT_HANDLE` (protocole valide mais non géré), `FRAGMENT` (fragment sans en-tête L4).
- Deux unions à lire selon le contexte : `ip_data` (`ipv4` si `version == 4`, `ipv6` si `version == 6`) et `data` (`tcp`, `udp`, `icmp` ou `arp`, valide seulement si `code == SUCCES`).
- Règle d'or du fichier : on vérifie qu'un octet existe avant de le lire, et chaque boucle a un nombre de tours maximum.

dict

- `Dict` : `size`, tableau de buckets `Entry **ptr`, compteurs, et six fonctions fournies à la création : `hash`, `compare`, `key_dup`, `key_free`, `value_dup`, `value_free`.
- `Entry` : `key`, `value`, le `hash` mémorisé, `next` (chaînage des collisions).
- Règle de propriété : si `key_dup` est fourni, le dict possède sa copie de la clé ; sinon il garde le pointeur brut. Pareil pour les valeurs.
- Rehash : la taille double quand `(n + 1) / size` atteint 3/4.
- Trois raccourcis de création : `create_dict_str`, `create_dict_int`, `create_dict_raw`. Les variantes `_hash` évitent de recalculer un hachage déjà connu.
- `dict_clear` libère les entrées, le tableau **et la structure `Dict` elle-même**.

dchained_list

- `DChainedList` : `head`, `tail`, `n_elements`, `max_element` (0 = illimité).
- Liste **intrusive** : les liens sont dans le `FlowEntry` (`lru_prev`, `lru_next`), donc aucun nœud supplémentaire n'est alloué.
- Convention actuelle : `head` = le plus ancien, `tail` = le plus récent. `ft_add` ajoute à la fin et l'éviction part de la tête.
- Opérations : `add_to_end`, `add_to_top`, `move_to_end`, `move_to_top`, `delete_from_end`, `delete_from_top`.

portmap

- `bits[1024]` de 64 bits = 65 536 ports, `count` (ports distincts vus), `refcount` (nombre de flux qui s'appuient sur cette IP).
- `portmap_add` renvoie 1 si le port est nouveau, 0 s'il était déjà vu. `portmap_test` lit un bit, `portmap_reset` remet tout à zéro.

flow

- **Clé canonique** : `FlowKeyIpv4` / `FlowKeyIpv6` contiennent `ip_min`, `ip_max`, `port_min`, `port_max`, `proto`. Leur premier champ `key` indique le type, ce qui permet de savoir quelle structure lire à partir d'un `void *`.
- **`FlowData`** : initiateur et receveur (IP + port, dans une union v4 / v6), `flag_count` (SYN, ACK, FIN, RST, PSH, URG, ECE, CWR), `n_pkt_ltr` / `n_pkt_rtl`, `octets_data_ltr` / `octets_data_rtl`, `proto`, `verdict` (`UNKNOW`, `BLOCK`, `LIMIT_REACHED`), `start_time` / `end_time` (réservés, à 0.0 pour l'instant).
- **Direction** : ltr = le paquet vient de l'initiateur (l'émetteur du premier paquet vu), rtl = il vient de l'autre bout.
- Convention du champ `v4` dans `flow.c` : 0 signifie IPv4 et 1 signifie IPv6.
- Fonctions : `build_flow_key`, `build_flow_entry`, `entry_update`, puis les fonctions fournies au dict (`flow_hash`, `flow_key_compare`, `flow_key_dup`, `flow_key_free`, `flow_value_dup`, `flow_value_free`) et `create_ft_dict`.

flow_table

- `FlowTable` : `size` (capacité), `nbr_elements`, `flow_dict`, `flow_dcl`, `port_per_ip`, `n_evicted`.
- API : `ft_create`, `ft_destroy`, `ft_get`, `ft_contains`, `ft_can_add`, `ft_add`, `ft_evict_oldest`, et surtout `ft_process`.
- Politique d'éviction actuelle : capacité fixe. Quand la table est pleine, on chasse le flux le plus ancien (la tête de la file). Il n'y a pas encore d'expiration par le temps.

7. Ce qui est prévu 🛠️

D'après le PDF « notes et décisions » du 10 octobre.

Brique	Idée	Statut
Moteur de capture	`engine_start` / `engine_stop`, un thread `AF_PACKET` par interface, chacun avec sa flow table et son ring	prévu (`main.c` est un stub)
Ring partagé	mmap nommé (`shm_open`), un écrivain C, un lecteur, compteurs `w` et `r`, lu par Python via ctypes (`ring_open`, `ring_read`)	prévu

Brique	Idée	Statut
Expiration par le temps	tant que la tête est trop vieille : écrire le flux dans le ring, le retirer du dict, libérer	pas codé (`start_time` / `end_time` valent 0.0)
Liste d'IP à ignorer	`ipset` à adressage ouvert, lecture sans lock, ajout par lot depuis Python	prévu
Détection de scan	par IP source, seuil d'environ 20 ports distincts dans une fenêtre	`PortMap` codée ; seuil et fenêtre à faire
Agrégateur	un petit événement `evt_t` par nouveau flux, pour voir un scan réparti sur plusieurs interfaces	prévu
Plus tard	balayage réseau, DDoS (table par IP cible), plusieurs process de détection	idée

8. Le plan du PDF face au code actuel ⚖️

Quelques endroits où le code a pris un chemin un peu différent du plan. Rien d'obligatoire : ce sont des choix à trancher à ton rythme.

Sujet	PDF (10 oct)	Code (11 oct)
Flow table	tableau de nœuds + liens par **indices** + liste de nœuds libres	un `FlowEntry` alloué par flux, liens par **pointeurs**
Ce que le dict stocke	un numéro de nœud	une copie du `FlowEntry` (`value_dup`)
Mémoire	bornée par un tableau de taille fixe	allocations dynamiques, bornée par `size`
Clé de flux	contient le numéro d'interface	pas de champ interface dans `FlowKey`
Suivi des ports par IP	`portset` de 32 ports, bitmap « plus tard »	`PortMap` bitmap déjà en place

9. Mini-glossaire 📚

- **Flux** : une conversation entre deux bouts (IP, ports, protocole), peu importe le sens.
- **Initiateur / receveur** : celui qui envoie le premier paquet vu / l'autre.
- **ltr / rtl** : paquet dans le sens initiateur vers receveur / dans le sens inverse.
- **LRU** : on garde en tête de file ce qui n'a pas bougé depuis le plus longtemps, pour le chasser en premier.
- **Clé canonique** : clé identique dans les deux sens de la conversation.
- **PortMap** : bitmap où chaque bit dit « ce port a été visité ».
- **refcount** : nombre de flux qui utilisent encore la `PortMap` d'une IP.